

05 — RAG Pipeline (Retrieval-Augmented Generation)

What it is: grounds LLM answers in your own documents instead of the model's training data — retrieve relevant chunks from HANA (sheet 04), inject them into the prompt, and force the LLM to answer only from that context.

Flow

user_query → retrieve_context() [HANA similarity search] → format_context()
→ inject into RAG_PROMPT → LLM → { answer, sources, context }

Code Pattern

```
RAG_PROMPT = ChatPromptTemplate.from_template("""
Answer the question using ONLY the context below.
If the context doesn't contain the answer, say you don't know – do not guess.
Context: {context}
Question: {user_query}
""")

def rag_query(conn, user_query, embedding_model, llm, table_name="VECTOR_STORE",
              top_k=5, min_score=0.0) -> dict:
    chunks = retrieve_context(conn, user_query, embedding_model, table_name, top_k, min_score)
    context = format_context(chunks)  # numbered chunks -> single string

    chain = RAG_PROMPT | llm | StrOutputParser()
    answer = chain.invoke({"context": context, "user_query": user_query})

    sources = [{"rank": c["rank"], "source_name": c["source_name"], "score": c["score"]}
               for c in chunks]  # citations for the UI

    return {"answer": answer.strip(), "context": context, "sources": sources}
```

Multi-Turn Memory (adds conversation awareness)

```
from langchain.memory import ConversationBufferWindowMemory

class ChatMemory:
    def __init__(self, window=5):
        self._memory = ConversationBufferWindowMemory(k=window, return_messages=True)

    def save(self, user_input, ai_output):
        self._memory.save_context({"input": user_input}, {"output": ai_output})

    def get_history_string(self) -> str:
        # -> "User: ...\nAssistant: ...\nUser: ...\nAssistant: ..."
        ...
```

Key Points

- The system prompt explicitly says **“if the context doesn’t contain the answer, say you don’t know”** — this is the #1 lever against hallucination in RAG.
- `min_score` filters out weakly-related chunks (e.g. 0.5) so noise doesn’t dilute the context.
- `format_context()` numbers each chunk so the LLM (and your UI) can cite [1], [2] sources.
- Conversation memory is a **sliding window** (k=5 turns) — old turns drop off automatically to bound token cost; this is a *client-side* concept, separate from HANA storage.
- Single-turn RAG (`rag_query`) and memory (`ChatMemory`) are separate, composable pieces — the chatbot template (sheet 06) is what wires them together.

Interview Q&A

Q: What’s the single most important prompt instruction for reducing RAG hallucination? A: Explicitly instructing the model to say “I don’t know” when the retrieved context doesn’t answer the question, rather than letting it fall back on parametric (training) knowledge.

Q: Why separate retrieval, formatting, and generation into different functions? A: Each is independently testable and swappable — you could replace the retriever (different vector store) or the prompt (different persona) without touching the other two.

Q: How would you evaluate whether a RAG answer is “grounded”? A: Check whether claims in the answer are traceable to the retrieved context/sources — manually, or automatically by asking a second LLM call to verify each answer sentence against the provided context (a “faithfulness” check).